COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 1, 2026	**DATE SUBMITTED:**	October 6, 2026
NAME:	Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 **Graphs**

1. PROCEDURE OUTPUT

What procedures did you perform?

PROCEDURE ILOA – Graph Implementation Utilizing Adjacency Matrix and Adjacency List

ILOA10p1.cpp > main()

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {

    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;

        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head

        return newNode;
    }

    int N; // number of nodes in the graph

public:

    adjNode** head; // adjacency list as array of pointers
```

```
33      // Constructor
34      DiaGraph(graphEdge edges[], int n, int N) {
35
36          // allocate new node
37          head = new adjNode*[N];
38          this->N = N;
39
40          // initialize head pointer for all vertices
41          for (int i = 0; i < N; ++i)
42              head[i] = nullptr;
43
44          // construct directed graph by adding edges to it
45          for (unsigned i = 0; i < n; i++) {
46
47              int start_ver = edges[i].start_ver;
48              int end_ver = edges[i].end_ver;
49              int weight = edges[i].weight;
50
51              // insert in the beginning
52              adjNode* newNode =
53                  getAdjListNode(end_ver, weight, head[start_ver]);
54
55              // point head pointer to new node
56              head[start_ver] = newNode;
57          }
58      }
```

```
60      // Destructor
61      ~DiaGraph() {
62
63          for (int i = 0; i < N; i++) {
64
65              adjNode* current = head[i];
66
67              while (current != nullptr) {
68                  adjNode* temp = current;
69                  current = current->next;
70                  delete temp;
71              }
72          }
73
74          delete[] head;
75      }
76  };
```

```
79  // print all adjacent vertices of given vertex
80  void display_AdjList(adjNode* ptr, int i)
81  {
82      while (ptr != nullptr) {
83
84          std::cout << "(" << i << ", "
85                    << ptr->val << ", "
86                    << ptr->cost << ") ";
87
88          ptr = ptr->next;
89      }
90
91      std::cout << std::endl;
92  }
```

```
95  // display adjacency matrix
96  void display_AdjMatrix(graphEdge edges[], int n, int N)
97  {
98      int matrix[10][10] = {0};
99
100     // place edge weights into the matrix
101     for (int i = 0; i < n; i++) {
102
103         int start = edges[i].start_ver;
104         int end = edges[i].end_ver;
105         int weight = edges[i].weight;
106
107         matrix[start][end] = weight;
108     }
109
110     std::cout << "\nGraph adjacency matrix\n";
111     std::cout << "(0 means there is no edge)\n\n";
112
113     std::cout << "    ";
114
115     for (int i = 0; i < N; i++)
116         std::cout << i << " ";
117
118     std::cout << std::endl;
119
120     for (int i = 0; i < N; i++) {
121
122         std::cout << i << "   ";
```

```
123
124             for (int j = 0; j < N; j++)
125                 std::cout << matrix[i][j] << " ";
126
127             std::cout << std::endl;
128         }
129     }
```

```
132 // graph implementation
133 int main()
134 {
135     // graph edges array
136     graphEdge edges[] = {
137
138         // (x, y, w) -> edge from x to y with weight w
139
140         {0, 1, 2},
141         {0, 2, 4},
142         {1, 4, 3},
143         {2, 3, 2},
144         {3, 1, 4},
145         {4, 3, 3}
146     };
147
148     int N = 6; // Number of vertices in the graph
149
150     // calculate number of edges
151     int n = sizeof(edges) / sizeof(edges[0]);
152
153     // construct graph
154     DiaGraph diagraph(edges, n, N);
155
156     // display adjacency matrix
157     display_AdjMatrix(edges, n, N);
158
159     // display adjacency list representation of graph
160     std::cout << "\nGraph adjacency list"
161               << std::endl;
```

```
163     std::cout << "(start_vertex, end_vertex, weight):"
164               << std::endl;
165
166     for (int i = 0; i < N; i++) {
167
168         // display adjacent vertices of vertex i
169         display_AdjList(diagraph.head[i], i);
170     }
171
172     return 0;
173 }
```

CODE OUTPUT

```
Graph adjacency matrix
(0 means there is no edge)

   0 1 2 3 4 5
0  0 2 4 0 0 0
1  0 0 0 0 3 0
2  0 0 0 2 0 0
3  0 4 0 0 0 0
4  0 0 0 3 0 0
5  0 0 0 0 0 0

Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

PROCEDURE ILO B.1 & B.2 – Graph Implementing Graph Traversal Algorithms (Breadth-First and Depth First Search)

graph.h

```cpp
#ifndef GRAPH_H
#define GRAPH_H

#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <stack>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src, dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const {
        return weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (size_t i = 1; i <= G.vertices(); i++) {
        os << i << ":\t";
        for (const auto &e : G.outgoing_edges(i))
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    size_t vertices() const {
        return V;
    }

    const std::vector<Edge<T>> &edges() const {
        return edge_list;
    }

    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    std::vector<Edge<T>> outgoing_edges(size_t v) const {
        std::vector<Edge<T>> result;
        for (const auto &e : edge_list)
            if (e.src == v)
                result.emplace_back(e);
        return result;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os,
                                    const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```cpp
// Create the graph from the activity
template <typename T>
Graph<T> create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (const auto &i : edges)
        for (const auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

```cpp
// Depth-First Search
template <typename T>
std::vector<size_t> depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1);

    while (!stack.empty()) {
        size_t current = stack.top();
        stack.pop();

        if (visited.find(current) == visited.end()) {
            visited.insert(current);
            visit_order.push_back(current);

            for (const auto &e : G.outgoing_edges(current))
                if (visited.find(e.dest) == visited.end())
                    stack.push(e.dest);
        }
    }
    return visit_order;
}
```

```
124  // Breadth-First Search
125  template <typename T>
126  std::vector<size_t> breadth_first_search(const Graph<T> &G, size_t dest) {
127      std::queue<size_t> queue;
128      std::vector<size_t> visit_order;
129      std::set<size_t> visited;
130
131      queue.push(1);
132
133      while (!queue.empty()) {
134          size_t current = queue.front();
135          queue.pop();
136
137          if (visited.find(current) == visited.end()) {
138              visited.insert(current);
139              visit_order.push_back(current);
140
141              for (const auto &e : G.outgoing_edges(current))
142                  if (visited.find(e.dest) == visited.end())
143                      queue.push(e.dest);
144          }
145      }
146      return visit_order;
147  }
148
149  #endif
```

main.cpp

main.cpp > main()

```
1   #include <iostream>
2   #include "graph.h"
3
4   using T = unsigned;
5
6   // Test DFS
7   void test_DFS() {
8       auto G = create_reference_graph<T>();
9
10      std::cout << "===== GRAPH =====" << std::endl;
11      std::cout << G << std::endl;
12
13      std::cout << "===== DEPTH-FIRST SEARCH =====" << std::endl;
14      std::cout << "DFS Order of vertices:" << std::endl;
15
16      auto dfs_visit_order = depth_first_search(G, 1);
17
18      for (auto v : dfs_visit_order)
19          std::cout << v << " ";
20
21      std::cout << "\n\n";
22  }
```

```cpp
// Test BFS
void test_BFS() {
    auto G = create_reference_graph<T>();

    std::cout << "===== BREADTH-FIRST SEARCH =====" << std::endl;
    std::cout << "BFS Order of vertices:" << std::endl;

    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order)
        std::cout << v << " ";

    std::cout << std::endl;
}

int main() {
    test_DFS();
    test_BFS();
    return 0;
}
```

CODE OUTPUT

```
Reyes\OneDrive\Documents\HOA10.1_REYES\"main
===== GRAPH =====
1:  {2: 2}, {5: 3},
2:  {1: 2}, {5: 5}, {4: 1},
3:  {4: 2}, {7: 3},
4:  {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:  {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:  {4: 4}, {7: 4}, {8: 1},
7:  {3: 3}, {6: 4},
8:  {4: 5}, {5: 3}, {6: 1},
9:

===== DEPTH-FIRST SEARCH =====
DFS Order of vertices:
1 5 8 6 7 3 4 2

===== BREADTH-FIRST SEARCH =====
BFS Order of vertices:
1 2 5 4 8 3 6 7
```

CODE ANALYSIS:

ILO A: Graph Implementation

The code implements a directed weighted graph using two common ways of representing a graph: an adjacency list and an adjacency matrix. First, the adjNode and graphEdge structures are used to store the connected vertices and their weights. These are then managed by the DiaGraph class, which uses pointers to create and organize the linked lists. After creating the graph, the display_AdjList() function shows the connections and weights of each vertex, while the display_AdjMatrix() function presents the same information using a two-dimensional array. In addition to that, the program uses dynamic memory and a destructor to properly remove the nodes when they are no longer needed which makes the code great. Through this process, I learned that a graph can be stored in different ways depending on how the data needs to be accessed. Furthermore, I also learned that different discussions from previous modules pave the way in creating this implementation using pointers, structures, arrays, linked lists, and dynamic memory that can work together in C++. Overall, this activity helped me understand the basic implementation of graphs and how different graph representations can be used to organize and display connections.

ILO B: DFS and BFS

Building on the graph implementation from ILO A, this code focuses on two ways of exploring a graph: Depth-First Search (DFS) and Breadth-First Search (BFS). The Edge structure first stores the starting vertex, destination vertex, and weight, while the Graph class organizes the vertices and edges. The create_reference_graph() function then creates the graph based on the connections given in the activity. From there, DFS uses a stack to explore one path as deeply as possible before going back to check other vertices. On the other hand, the BFS uses a queue to visit the closest vertices first before moving to the next level. Both methods also use a set to keep track of visited vertices, which prevents the program from visiting the same vertex repeatedly. The test_DFS() and test_BFS() functions then show the order of the visited vertices, starting from vertex 1. Through this implementation, I learned how DFS and BFS work and how the use of a stack or queue changes the order in which vertices are visited. I also gained more practice using vectors, sets, maps, stacks, queues, classes, and templates in C++. Overall, the activity helped me connect the basic graph concepts from ILO A to actual graph traversal and understand how different methods can be used to explore a graph.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILOC – Graph Implementation, Operations and Traversal Methods

1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore another vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.**

- The most helpful algorithm to use in this case is Depth-First Search (DFS). DFS starts from one vertex and explores as far as possible along one path before backtracking. After reaching the end of a path, it goes back and explores the other available vertices from the same point. This matches the situation described because the person wants to finish one path first before exploring another.

2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**

- The equivalent of DFS in tree traversal is Preorder Traversal. Both start at the current node and explore its children before moving to other parts of the tree. In Preorder, the order is Root → Left → Right, while DFS follows a similar deep exploration process. Both can also be implemented using a stack or recursion, making them closely related in how they visit nodes.

SIMPLE COMPARISON:

Depth-First Search (DFS): Start → Go Deeper → Backtrack → Explore Next

Preorder Traversal: Start at Root → Visit Root → Visit Left → Visit Right

3. **In the performed code, what data structure is used to implement the Breadth First Search?**

- The data structure used to implement Breadth-First Search (BFS) in the performed code is a queue. It follows the First-In, First-Out (FIFO) principle, meaning the vertex that enters the queue first is also the first one to be processed. This allows BFS to visit vertices level by level.

4. **How many times can a node be visited in the BFS?**

- In the performed BFS code, a node can be visited only once. The set named visited keeps track of the nodes that have already been visited. This prevents the algorithm from processing the same node repeatedly, even if there are multiple connections leading to it.

CONCLUSION/LEARNINGS/ANALYSIS

Overall, the activities helped me understand how graphs are implemented, represented, and traversed in C++. I learned how adjacency lists and adjacency matrices can be used to store graph connections, while DFS and BFS provide different ways to explore the vertices. In addition, I also learned that DFS uses a stack to explore deeply, while BFS uses a queue to visit vertices level by level. Furthermore, the supplementary activity also paved the way for me to understand the connection between DFS and tree traversal, particularly Preorder Traversal, as well as the importance of keeping track of visited nodes. Through these given activities, I gained a better understanding of graph operations, traversal methods, and how different data structures work together in implementing graph algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: Code Assistance and Explanation of Concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI paved the way for me to understand the concept, helped me debug and reconstruct my code in accordance to what the laboratory is looking for. Other than that, AI was not used to replace my main idea of how I wanted to implement the code and output.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.